A ARKA-Lite

EQUIPO ARKA-LITOS · UPSKILLING BACKEND JAVA

Del endpoint al clúster

Cómo construimos dos microservicios con sus datos, su plataforma y su operación.

2 microservicios	**2** bases PostgreSQL	**1** comando levanta todo	**0** avisos perdidos o duplicados

VERSIÓN TÉCNICO · STAFF DE TECNOLOGÍA



Qué vamos a ver

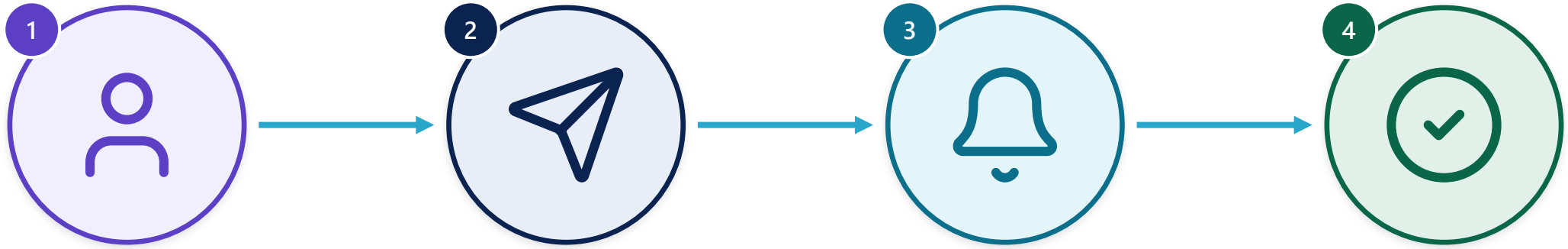
Las mismas secciones que todos los equipos; nosotros vamos a fondo en datos, interconexión y DevOps.



1	2	3	4	5	6	7
Fundamentos	**Arquitectura**	**Base de datos**	**Interconexión de servicios**	**DevOps: Docker y Kubernetes**	**Observabilidad**	**Demostración con la UI**
VISTA GENERAL	VISTA GENERAL	A FONDO	A FONDO	A FONDO	VISTA GENERAL	EN VIVO
Primero el qué · Código mantenible · Git y documentación	Microservicios · Una base por servicio · Hexagonal en una frase	Modelo de datos · ORM con JPA · Cadena de conexión	Síncrono y asíncrono · Patrón Outbox · Idempotencia	Docker y Compose · Kubernetes · CI/CD con GitHub Actions	Logs centralizados · Trazas · Salud con Actuator	Crear y enviar · Una caída en vivo · Reentregar

Primero el qué: el negocio en una lámina

Antes de escribir código, entendimos qué tiene que pasar, quién lo hace y qué no se permite.



Quien solicita

Registra qué necesita y de qué tipo.

La envía

Cuando está lista.

El sistema avisa

A quien debe atenderla.

Alguien decide

La aprueba o la rechaza.

si se salta un paso

No se permite

Enviar dos veces o decidir sobre un borrador: el sistema explica por qué.

Código mantenible: buenas prácticas

Después, el cómo: seis prácticas que hacen que un cambio toque un solo lugar, en cualquier lenguaje.

Una tarea por pieza

Quien recibe la petición no decide; quien decide no guarda.

Piezas que se conectan

Cada pieza recibe lo que necesita: se cambia o se prueba sola.

Reglas en un solo lugar

El negocio vive junto, lejos de la pantalla y de la base.

Errores con respuesta clara

Siempre el mismo formato: 400, 404 o 409 con su motivo.

Nombres del negocio

Solicitud, enviar, aprobar: el código se lee como el proceso.

Cada regla con su prueba

Una prueba automática avisa si un cambio rompió algo.

Código mantenible: buenas prácticas · el código

servicio-solicitudes/…/dominio/ServicioDeSolicitudes.java

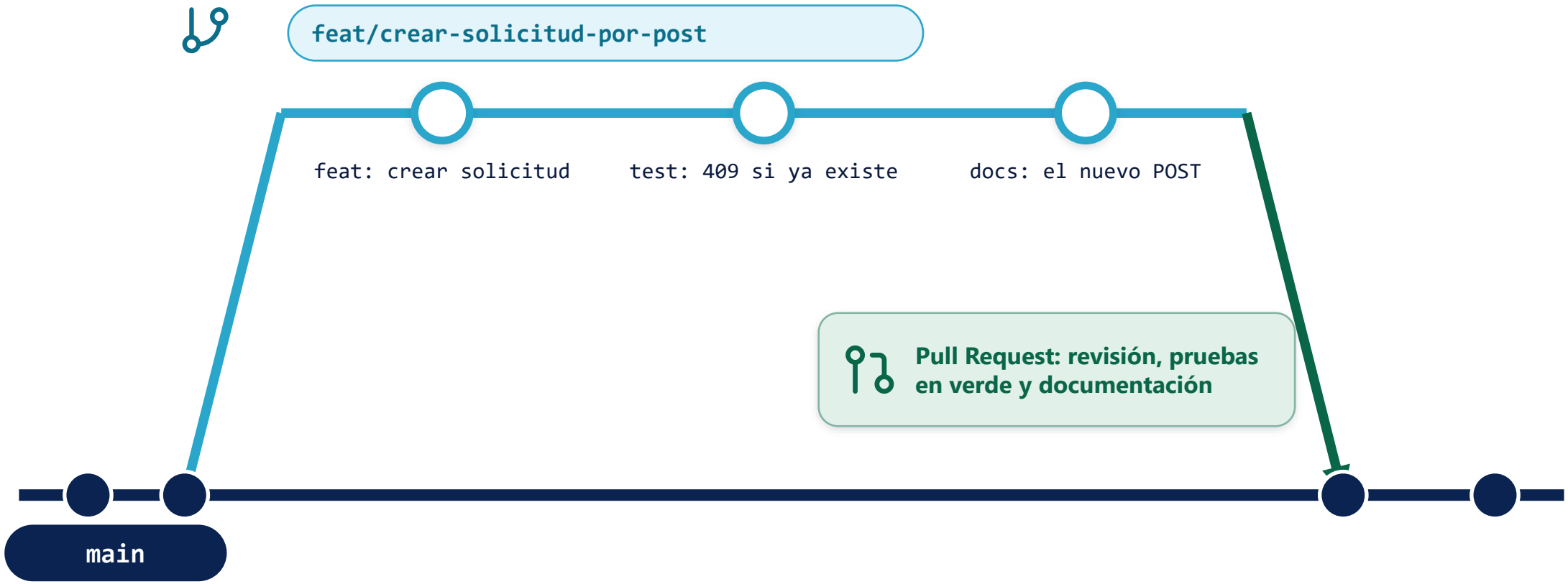
```
public class ServicioDeSolicitudes {      // sin anotaciones de Spring

    private final RepositorioDeSolicitudes repositorio;       // interfaces,
    private final PublicadorDeEventos publicador;             // no clases concretas
    private final RegistradorDeEventosSalientes registrador;

    public ServicioDeSolicitudes(RepositorioDeSolicitudes repositorio,
                                 PublicadorDeEventos publicador,
                                 RegistradorDeEventosSalientes registrador) {
        this.repositorio = repositorio;   // todo llega por el constructor
        this.publicador = publicador;
        this.registrador = registrador;
    }
}
```

Trabajo en equipo: Git y documentación

Todo cambio entra revisado, probado y documentado.



Fundamentos: lo que nos llevamos a cualquier proyecto

LO QUE APRENDIMOS

6

prácticas que sirven en cualquier lenguaje

4

respuestas claras: 201, 400, 404 y 409

Primero el qué, después el cómo

Qué tiene que pasar, quién lo hace y qué no se permite.

VALOR

Reglas claras antes de elegir la tecnología.

Separar responsabilidades paga pronto

Pantalla, reglas y datos en piezas distintas.

VALOR

Un cambio de regla toca un solo lugar.

Las reglas no dependen de la herramienta

El negocio no sabe si detrás hay una base, una web o un framework.

VALOR

Se prueba rápido y sobrevive a un cambio de tecnología.

Todo cambio entra revisado

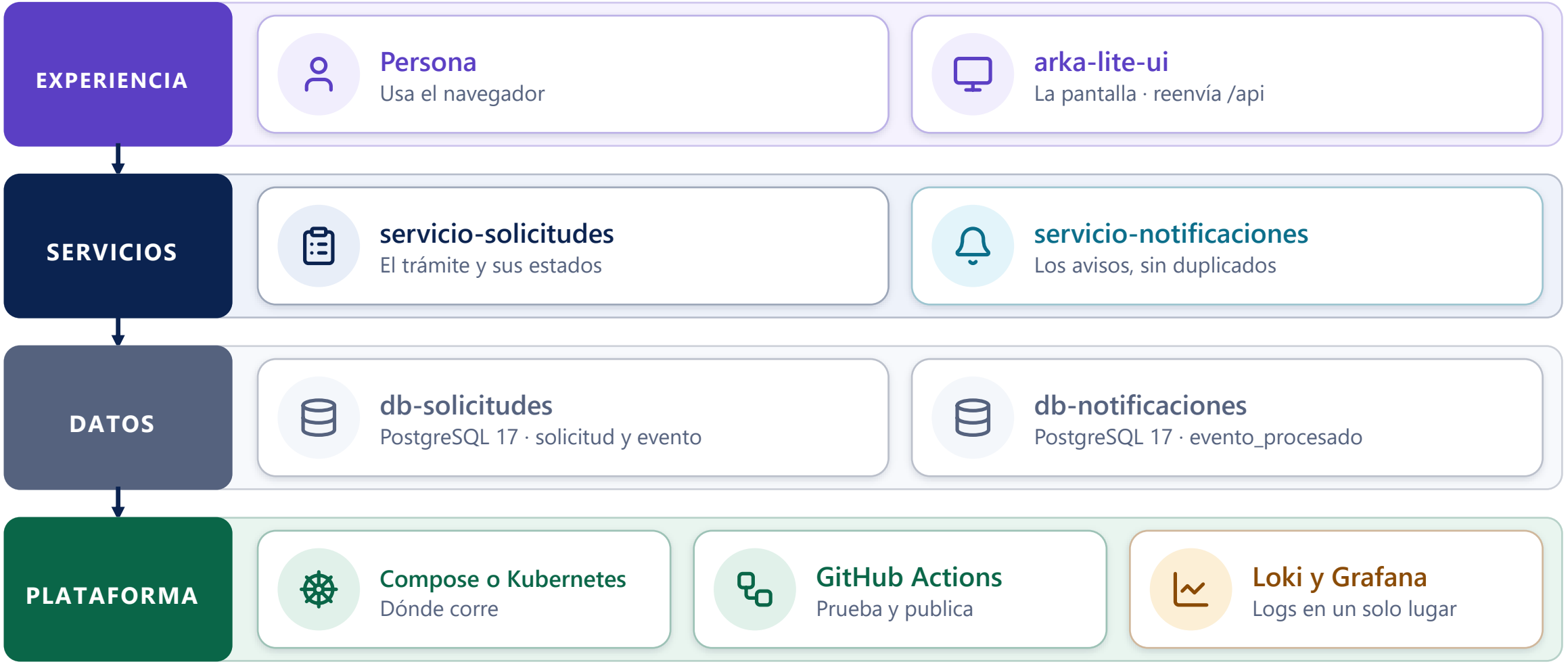
Ramas cortas, revisión, pruebas y documentación juntas.

VALOR

Menos sorpresas en producción.

El sistema de un vistazo

Cada capa tiene una sola responsabilidad.



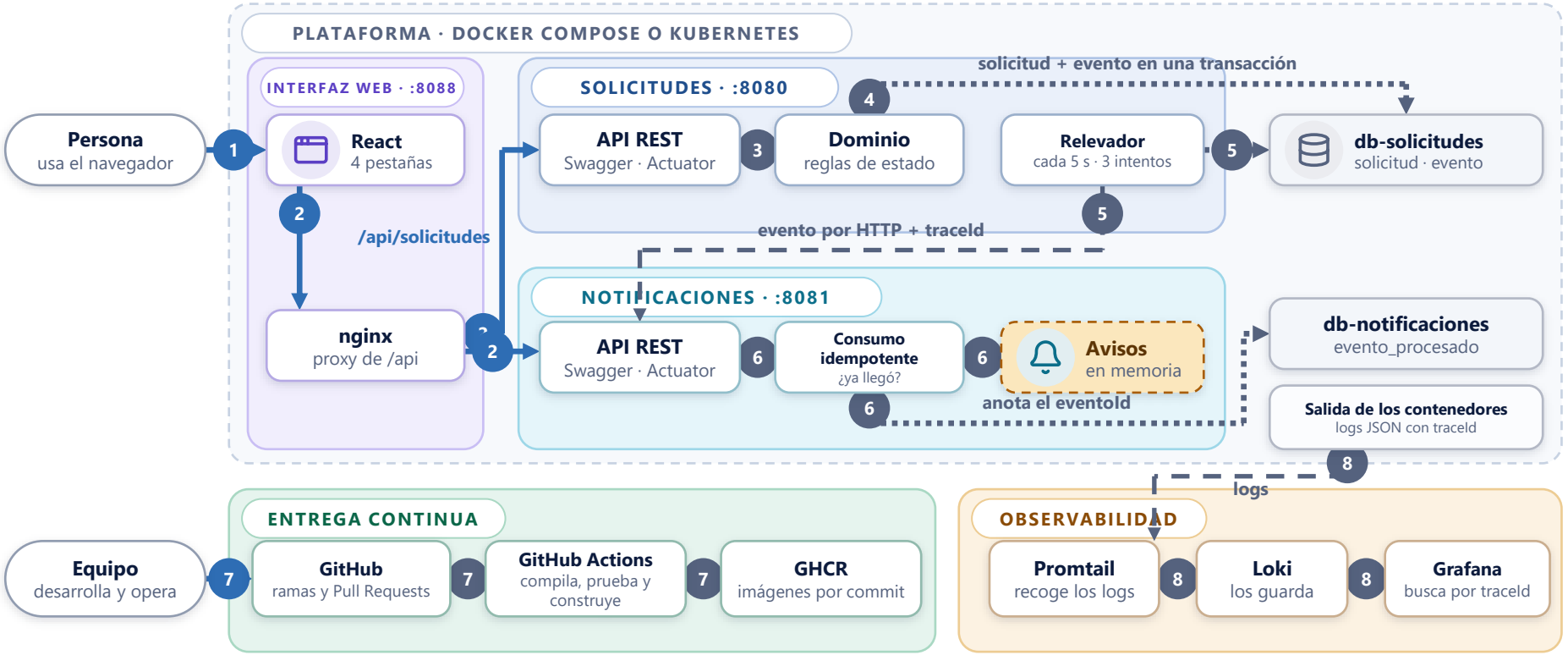
Cómo funciona en la práctica

Seis conexiones, y ninguna une las dos bases.



Ninguna flecha une las dos bases: cada servicio es dueño de sus datos.

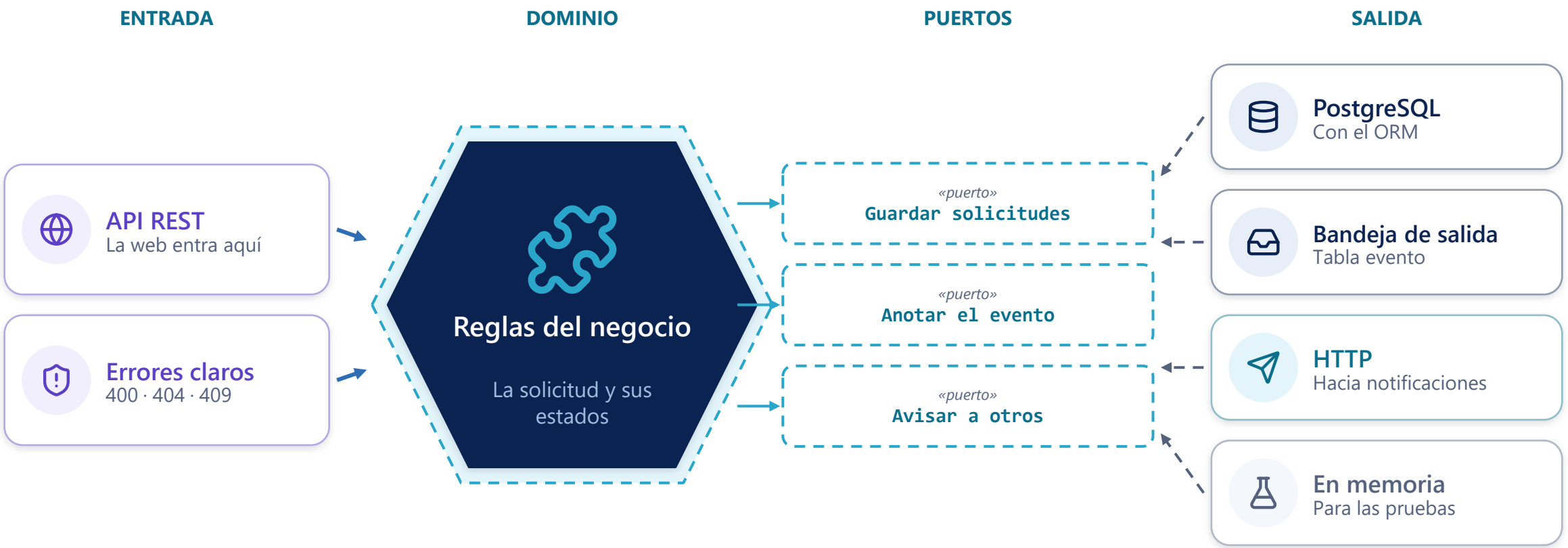
La arquitectura completa



1. La persona usa la interfaz en su navegador: React muestra las cuatro pestañas.
2. nginx recibe /api y reenvía cada ruta al servicio que corresponde.
3. La API de solicitudes recibe la petición y el dominio aplica las reglas.
4. En una sola transacción se guardan la solicitud y su evento pendiente.
5. Cada 5 segundos, el relevador lee los pendientes y los entrega por HTTP, con el traceId.
6. Notificaciones revisa si el evento ya llegó; si es nuevo, crea el aviso y anota su número.
7. Cada cambio en GitHub pasa por GitHub Actions, que prueba y publica las imágenes en GHCR.
8. Los logs de todos los contenedores llegan a Grafana a través de Promtail y Loki.

Por dentro de cada servicio: hexagonal

Una forma de ordenar el código por dentro; no cambia cómo se despliega.



Piénsenlo así: Como un enchufe: el aparato funciona con cualquier toma que cumpla la norma, sin saber de qué planta viene la electricidad.

Por dentro de cada servicio: hexagonal · el código

servicio-solicitudes/…/config/ConfiguracionDominio.java

```
@Bean
ServicioDeSolicitudes servicioDeSolicitudes(
        RepositorioDeSolicitudes repositorio,      // RepositorioJpa (@Primary)
        PublicadorDeEventos publicador,            // PublicadorConReintento (@Primary)
        RegistradorDeEventosSalientes registrador) {
    return new ServicioDeSolicitudes(repositorio, publicador, registrador);
}
```

¿Monolito o microservicios?

Separar se decide por el negocio, no por moda.

Conviene un monolito

- Las partes cambian juntas
- Se publica todo junto
- La carga es pareja
- Un equipo pequeño lo mantiene

vs

Conviene separar

- Responsabilidades claramente distintas
- Una parte necesita salir sola
- Una parte necesita más recursos
- Un equipo dueño de cada parte

¿Monolito o microservicios? · el código

servicio-*/…/eventos/SolicitudEnviada.java

```
// Copiado en ambos servicios: es el contrato HTTP
public record SolicitudEnviada(String eventoId, String id, String tipo) {}
```

Arquitectura: lo que aprendimos



LO QUE APRENDIMOS

2

servicios, cada uno con su base

0

tablas compartidas entre servicios



Cada servicio, dueño de sus datos

Nadie lee ni escribe en la base del otro.

VALOR

Cada uno cambia sin romper al otro.



Entre servicios viajan datos

Un evento pequeño con su número: SolicitudEnviada.

VALOR

Un acuerdo claro y fácil de revisar.



Hexagonal ordena por dentro

No cambia el despliegue: un contenedor por servicio.

VALOR

Cambiar la base o la web sin tocar las reglas.



Separar tiene un costo

Más red, dos bases y fallas parciales.

VALOR

Separar solo cuando el negocio lo pide.

Dos bases, una por servicio

Cada microservicio tiene su propia base, y las dos corren en PostgreSQL.

2

bases de datos

db-solicitudes y db-notificaciones: una por servicio.

17

PostgreSQL

El mismo motor y la misma versión en las dos.

0

JOIN entre bases

Ni consultas ni llaves foráneas que crucen de una a otra.

1

disco por base

En Kubernetes, los datos sobreviven aunque el contenedor se reinicie.

La estructura de las bases

Tres tablas en dos bases, y una sola cosa cruza la frontera: el número del evento.

db-solicitudes

PostgreSQL 17 · schema.sql

solicitud

- **id** PK · VARCHAR(20)
- # **tipo** VARCHAR(20)
- # **estado** CHECK: 4 estados

evento

- **id** PK · BIGSERIAL
- **solicitud_id** FK a solicitud
- # **tipo** SolicitudEnviada
- # **evento_id** UUID
- # **procesado** false = pendiente

evento_id por HTTP

sin llaves foráneas entre bases

db-notificaciones

PostgreSQL 17 · Hibernate

evento_procesado

- **evento_id** PK
- # **uno por evento recibido**

¿Por qué importa? Cada servicio cambia su esquema sin coordinarse con el otro; solo el número del evento es compartido.

Un esquema que se cuida solo

Sin herramientas de migración, pero sin sorpresas al arrancar.

schema.sql idempotente

CREATE y ALTER … IF NOT EXISTS: se puede correr mil veces y también actualiza bases viejas.

Hibernate valida

ddl-auto=validate: si una entidad no coincide con la tabla, el servicio no arranca.

La base también cuida reglas

Un CHECK solo admite BORRADOR, ENVIADA, APROBADA o RECHAZADA.

Índice parcial

idx_evento_pendiente solo indexa las filas con procesado = false.

Un esquema que se cuida solo · el código

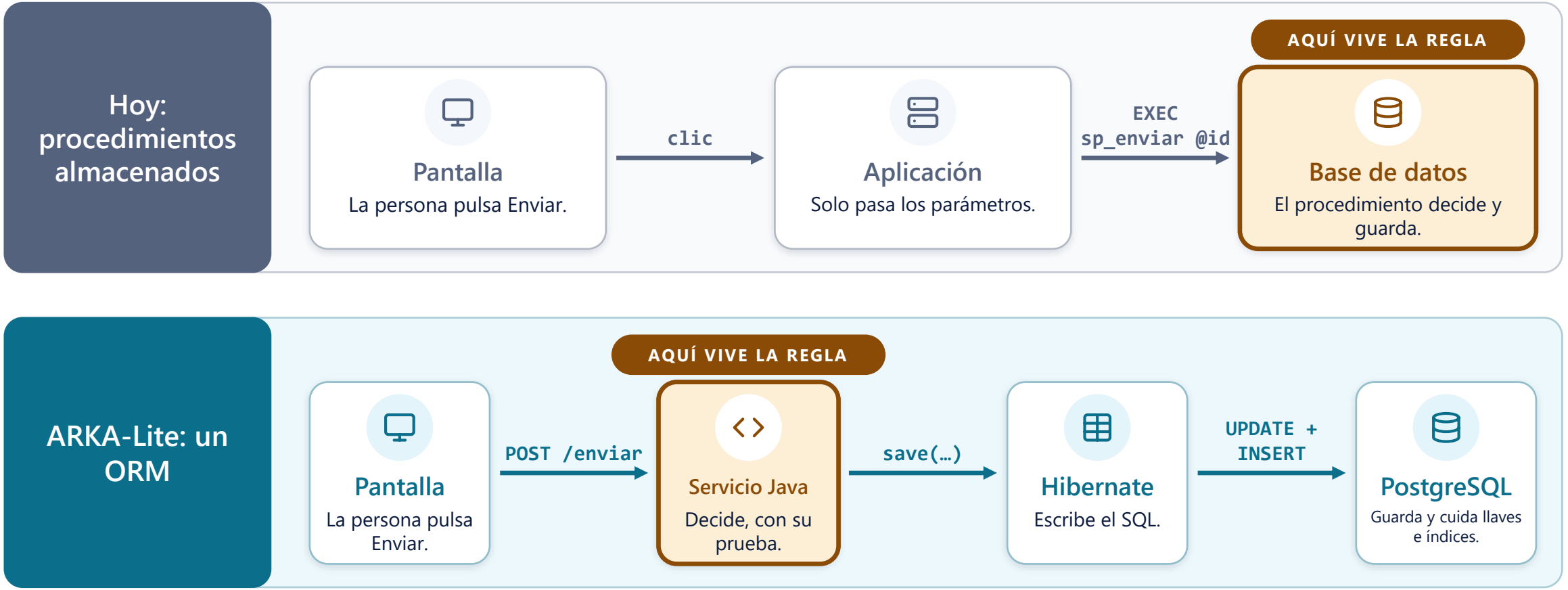
servicio-solicitudes/src/main/resources/schema.sql

```
ALTER TABLE evento ADD COLUMN IF NOT EXISTS
    evento_id UUID NOT NULL DEFAULT gen_random_uuid();
ALTER TABLE evento ADD COLUMN IF NOT EXISTS
    procesado BOOLEAN NOT NULL DEFAULT TRUE;
ALTER TABLE evento ALTER COLUMN procesado SET DEFAULT FALSE;

CREATE INDEX IF NOT EXISTS idx_evento_pendiente
    ON evento (procesado) WHERE procesado = FALSE;
```

Del procedimiento almacenado al ORM

La diferencia no es la base de datos: es dónde vive la regla del negocio.



¿Por qué importa? No es todo o nada: las reglas van al código y los reportes o consultas pesadas pueden seguir en SQL.

La misma regla, escrita de las dos formas

Regla del negocio: solo se puede enviar una solicitud que está en borrador.

Hoy: en un procedimiento

```
CREATE PROCEDURE sp_enviar @id VARCHAR(20) AS
BEGIN
  IF (SELECT estado FROM solicitud
      WHERE id = @id) <> 'BORRADOR'
    THROW 50001, 'Solo se envía en BORRADOR', 1;
  UPDATE solicitud SET estado = 'ENVIADA'
   WHERE id = @id;
END
```

- La regla y el dato viven juntos en la base
- Probarla exige una base con datos
- Cambiarla es desplegar en la base

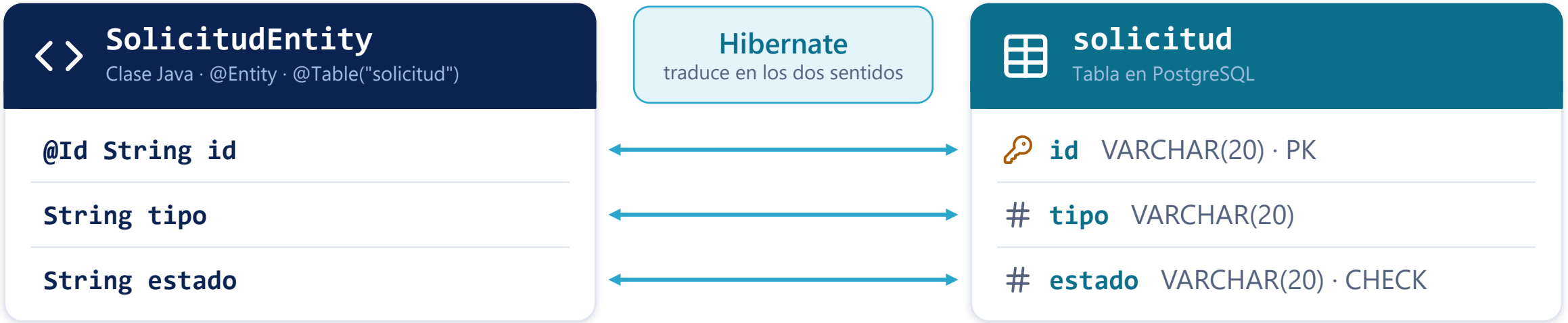
ARKA-Lite: en el código

```
public Solicitud enviar() {
    if (estado != Estado.BORRADOR)
        throw new TransicionNoPermitida(
            "solo se puede enviar en BORRADOR");
    return new Solicitud(id, tipo, Estado.ENVIADA);
}
// Hibernate escribe el SQL:
// UPDATE solicitud SET estado = ? WHERE id = ?
```

- La regla vive en el código, con su prueba
- Se prueba en milisegundos, sin base
- La base sigue cuidando llaves, CHECK e índices

El ORM por dentro: una clase es una tabla

Hibernate lee la clase, sabe a qué columna va cada campo y escribe el SQL.



LO QUE ESCRIBE EL CÓDIGO → EL SQL QUE HIBERNATE ENVÍA A POSTGRESQL

`jpa.save(nueva)`	`INSERT INTO solicitud (id, tipo, estado) VALUES (?, ?, ?)`
`jpa.findById("CAM-002")`	`SELECT id, tipo, estado FROM solicitud WHERE id = ?`
`jpa.save(enviada)`	`UPDATE solicitud SET tipo = ?, estado = ? WHERE id = ?`

El ORM por dentro: una clase es una tabla · el código

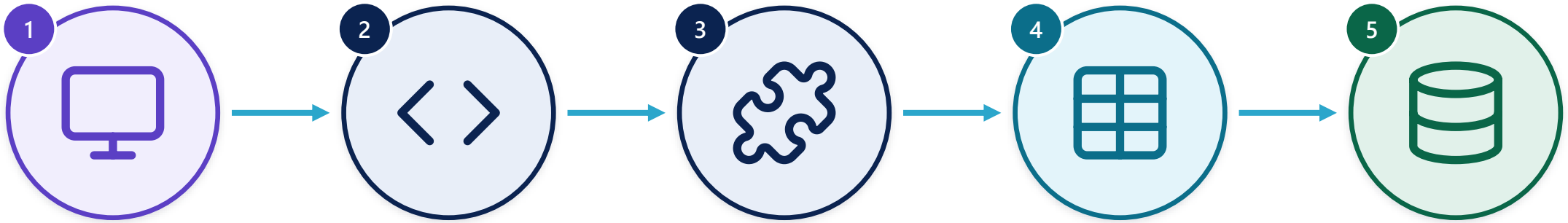
servicio-solicitudes/…/adaptadores/persistencia/SolicitudEntity.java

```java
@Entity
@Table(name = "solicitud")          // esta clase = la tabla solicitud
public class SolicitudEntity {
    @Id
    private String id;               // columna id (PK)
    private String tipo;             // columna tipo
    private String estado;           // columna estado
}

// Sin SQL escrito: Spring Data genera SELECT, INSERT y UPDATE
public interface SolicitudJpaRepository
        extends JpaRepository<SolicitudEntity, String> {}
```

El viaje de un dato: de la pantalla a la fila

El mismo dato cambia de forma en el camino: JSON, objeto, SQL y fila.



1 Pantalla

Manda JSON: SOL-100, tipo Consulta.

2 Controlador

Convierte el JSON en un objeto.

3 Reglas

Revisa que el id no exista. Sin SQL.

4 Hibernate

Objeto → INSERT con parámetros.

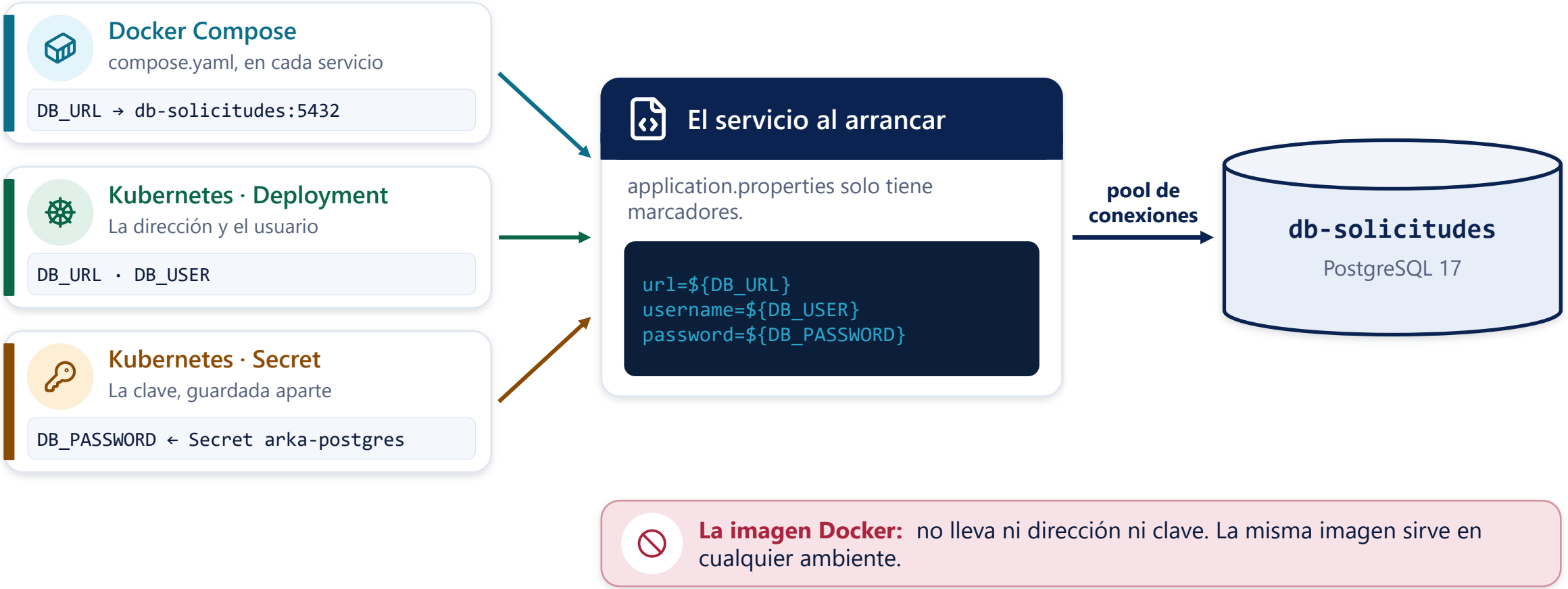
5 PostgreSQL

Guarda la fila y hace COMMIT.

Piénsenlo así: Como un traductor: el código habla Java, la base habla SQL, y el ORM traduce en los dos sentidos.

¿Dónde está la cadena de conexión?

Ni en el código ni en la imagen: la pone el ambiente donde corre la aplicación.



¿Por qué importa? Cambiar de servidor o rotar una clave no obliga a recompilar ni a tocar el código.

¿Dónde está la cadena de conexión? · el código

servicio-solicitudes/src/main/resources/application.properties

```
# Conexión: el ambiente da los valores; después de : va el valor local
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/db-solicitudes}
spring.datasource.username=${DB_USER:arka}
spring.datasource.password=${DB_PASSWORD:arka}

# Esquema: schema.sql en cada arranque y Hibernate solo valida
spring.sql.init.mode=always
spring.jpa.hibernate.ddl-auto=validate
```

Una transacción: todo o nada

@Transactional es el BEGIN … COMMIT de siempre: la solicitud y su aviso se guardan juntos o no se guarda nada.



Pool

HikariCP presta una de sus conexiones.

BEGIN

Empieza la transacción.

UPDATE

La solicitud pasa a ENVIADA.

INSERT

El aviso queda pendiente en evento.

COMMIT

Las dos cosas quedan guardadas.

ROLLBACK

No queda nada a medias: ni la solicitud ni el aviso.

Base de datos: lo que aprendimos

LO QUE APRENDIMOS

3

tablas en dos bases

0

consultas que crucen de una base a otra

El ORM no elimina el trabajo de base de datos

Escribe el SQL rutinario; índices, CHECK y transacciones siguen siendo diseño.

VALOR

Reglas probadas en código y una base que se cuida sola.

La cadena de conexión no vive en el código

Variables en Compose y un Secret en Kubernetes.

VALOR

Cambiar de servidor o de clave sin recompilar.

La transacción local es la más confiable

Lo que cambia junto se guarda en el mismo COMMIT.

VALOR

Nunca quedan datos a medias.

El esquema también es código

schema.sql versionado, idempotente y validado al arrancar.

VALOR

Despliegues repetibles y errores visibles al inicio.

Quién habla con quién, y cómo

Todo viaja por HTTP con JSON, y cada pieza encuentra a la otra por su nombre.

CONEXIÓN	CÓMO SE HABLA	POR QUÉ ASÍ
Navegador → UI	HTTP a localhost:8088; nginx sirve la página	El navegador nunca conoce las direcciones internas
UI → servicios	nginx reenvía /api/solicitudes y /api/notificaciones	Cambiar una dirección no obliga a tocar la UI
solicitudes → notificaciones	POST del evento SolicitudEnviada en JSON, desde el relevador	Si notificaciones cae, solicitudes sigue funcionando
Cómo se encuentran	Por nombre: la red de Compose o un Service de Kubernetes	Sin IPs fijas: funciona igual en cualquier ambiente
Seguimiento	El traceId viaja en la cabecera de cada llamada HTTP	Una sola búsqueda en Grafana sigue todo el recorrido

Quién habla con quién, y cómo · el código

servicio-solicitudes/…/adaptadores/salida/PublicadorConReintento.java

```
public PublicadorConReintento(
        @Value("${notificaciones.url}") String url,   // NOTIFICACIONES_URL
        RestClient restClientTrazada) {                // propaga el traceId
    this.url = url;
    this.rest = restClientTrazada;
}

// en publicar(evento): hasta 3 intentos
rest.post()
    .uri(url)          // http://servicio-notificaciones:8080/eventos/solicitud-enviada
    .body(evento)      // {"eventoId": "…", "id": "…", "tipo": "…"}
    .retrieve()
    .toBodilessEntity();
```

Síncrono o asíncrono

Lo que la persona necesita ver se responde ya; lo que puede esperar se avisa después.

Síncrono

- La UI llama y espera que se complete el proceso.
- Respuesta inmediata
- Si el servicio se cae, la persona ve error

vs

Asíncrono

- Servicio solicitudes envía evento a servicio notificaciones y continúa con su proceso
- Respuesta puede demorar
- Si notificaciones se cae, el evento espera

Piénsenlo así: Síncrono es una llamada: me quedo en la línea hasta que me responden. Asíncrono es un correo: lo envío y sigo con lo mío.

Patrón Outbox: guardar primero, avisar después

La solicitud y su evento se guardan en la misma transacción; el relevador avisa después.



Enviar

Solicitud ENVIADA + evento pendiente, en una transacción.

Bandeja de salida

El evento espera en la tabla evento.

Relevador

Cada 5 s entrega los pendientes, hasta 3 intentos.

si falla

Sigue pendiente

Mismo eventoId; se reintenta en el siguiente ciclo.

Notificaciones

Lo recibe y el evento queda procesado.

Piénsenlo así: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

¿Por qué importa? Si notificaciones está caído, ningún aviso se pierde: queda pendiente hasta que vuelve.

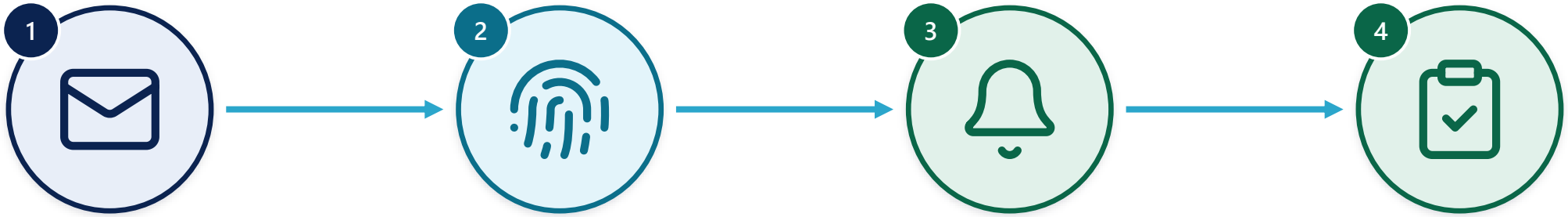
Patrón Outbox: guardar primero, avisar después · el código

servicio-solicitudes/…/dominio/ServicioDeSolicitudes.java

```
@Transactional
public Solicitud enviar(String id) {
    Solicitud enviada = buscar(id).enviar();
    repositorio.guardar(enviada);
    registrador.registrar(UUID.randomUUID().toString(),
            enviada.id(), "SolicitudEnviada");
    return enviada;
}
```

Idempotencia: el mismo evento, un solo aviso

Repetir la entrega no multiplica el efecto.



Llega el evento

Trae su eventId.

¿Ya lo recibimos?

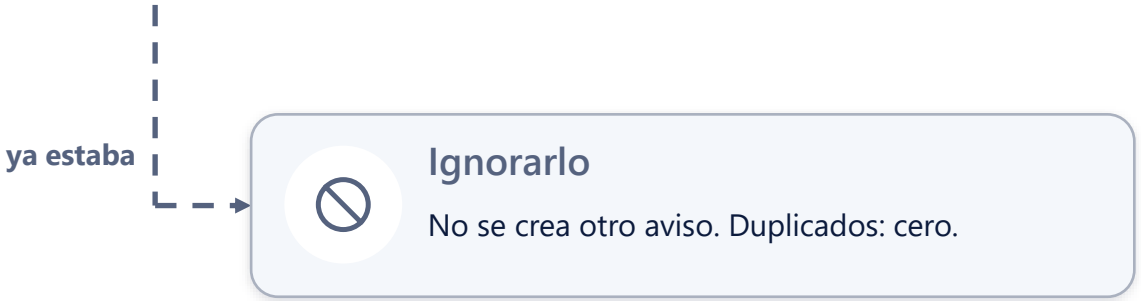
Se busca en evento_procesado.

Crear el aviso

Solo si el número es nuevo.

Anotar el número

En la misma transacción.



Piénsenlo así: Como un portero con lista de invitados: si alguien ya entró, no lo vuelve a anotar.

Idempotencia: el mismo evento, un solo aviso · el código

servicio-notificaciones/…/adaptadores/entrada/ConsumoIdempotente.java

```
@Transactional
public void alRecibir(SolicitudEnviada e) {
    if (procesados.existsById(e.eventoId())) {
        return; // ya procesado: idempotente
    }
    servicio.alRecibirSolicitudEnviada(e);
    procesados.save(new EventoProcesadoEntity(e.eventoId()));
}
```

Interconexión: lo que aprendimos



LO QUE APRENDIMOS

0

avisos perdidos

0

avisos duplicados



Síncrono para ver, asíncrono para avisar

La UI espera la respuesta; el evento se entrega después.

VALOR

Una caída parcial no detiene a quien usa el sistema.



Guardar primero, avisar después

Solicitud y evento en la misma transacción; el relevador entrega.

VALOR

Un servicio caído no hace perder información.



Todo reintento necesita idempotencia

Un número de evento fijo y la lista de los ya recibidos.

VALOR

Reintentar sin miedo a duplicar.



El contrato es lo único compartido

SolicitudEnviada: eventoId, id y tipo, igual en los dos lados.

VALOR

Equipos independientes con un acuerdo claro.

¿Qué es DevOps? Del código a la operación

No es una herramienta: es un solo camino, automático y repetible, del código a producción y de vuelta.



Docker y Kubernetes: no compiten, se complementan

Docker construye la caja; Kubernetes la pone a correr y la cuida.

Docker: empaqueta

- Mete la app con Java y todo lo que usa
- Crea una imagen que no cambia
- Corre igual en una laptop o en un servidor
- Compose levanta varias cajas juntas

+

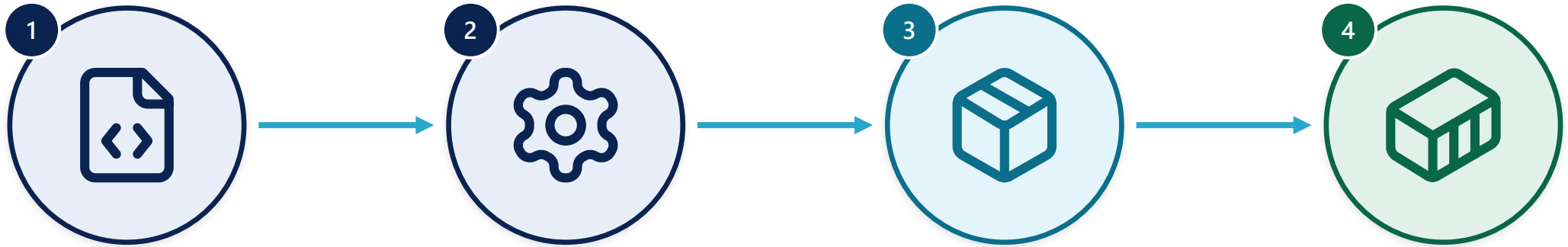
Kubernetes: administra

- Decide dónde corre cada caja
- Si una se cae, levanta otra
- Le da a cada servicio un nombre fijo
- Puede correr más copias si hay carga

Piénsenlo así: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

De código a contenedor

Una receta que da el mismo resultado en cualquier computadora.



Dockerfile

La receta: qué instalar y qué copiar.

Etapa build

Compila el JAR con Maven.

Imagen

Solo Java y el JAR: más liviana.

Contenedor

Corre con un usuario sin privilegios.

De código a contenedor · el código

servicio-solicitudes/Dockerfile

```
FROM maven:3.9-eclipse-temurin-26 AS build
RUN mvn -q clean package -DskipTests

FROM eclipse-temurin:26-jre
COPY --from=build /build/target/*.jar app.jar
RUN useradd -r -u 1001 appuser1
USER appuser1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Todo el sistema con un comando

Ocho contenedores, una red y la configuración por variables de entorno.

```
>_ $ docker compose up --build -d
```

arka-lite-ui

:8088 · la interfaz y su proxy

servicio-solicitudes

:8080 · el trámite

servicio-notificaciones

:8081 · los avisos

db-solicitudes

PostgreSQL 17 · solo red interna

db-notificaciones

PostgreSQL 17 · :5433

promtail

recoge los logs

loki

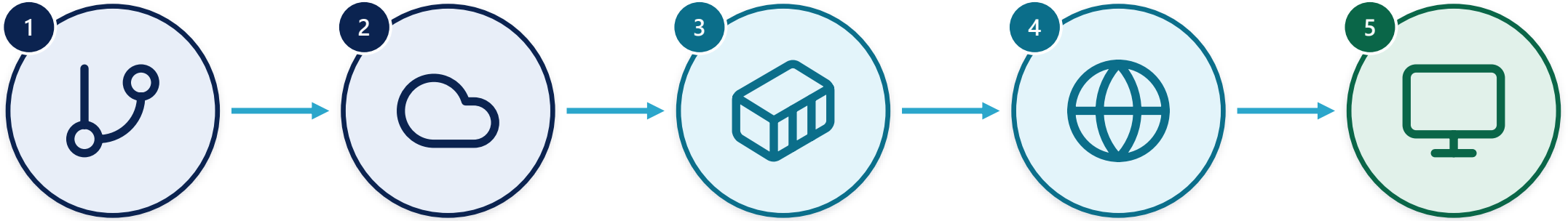
:3100 · guarda los logs

grafana

:3000 · busca en los logs

Sin instalar nada: GitHub Codespaces

El sistema completo, levantado en la nube desde una rama: así hacemos la demostración.



Una rama

Code › Codespaces › Crear.

Máquina en la nube

4 núcleos y Docker adentro.

levantar.sh

Construye y levanta los 8 contenedores.

Puertos https

UI, Grafana y Swagger, privados.

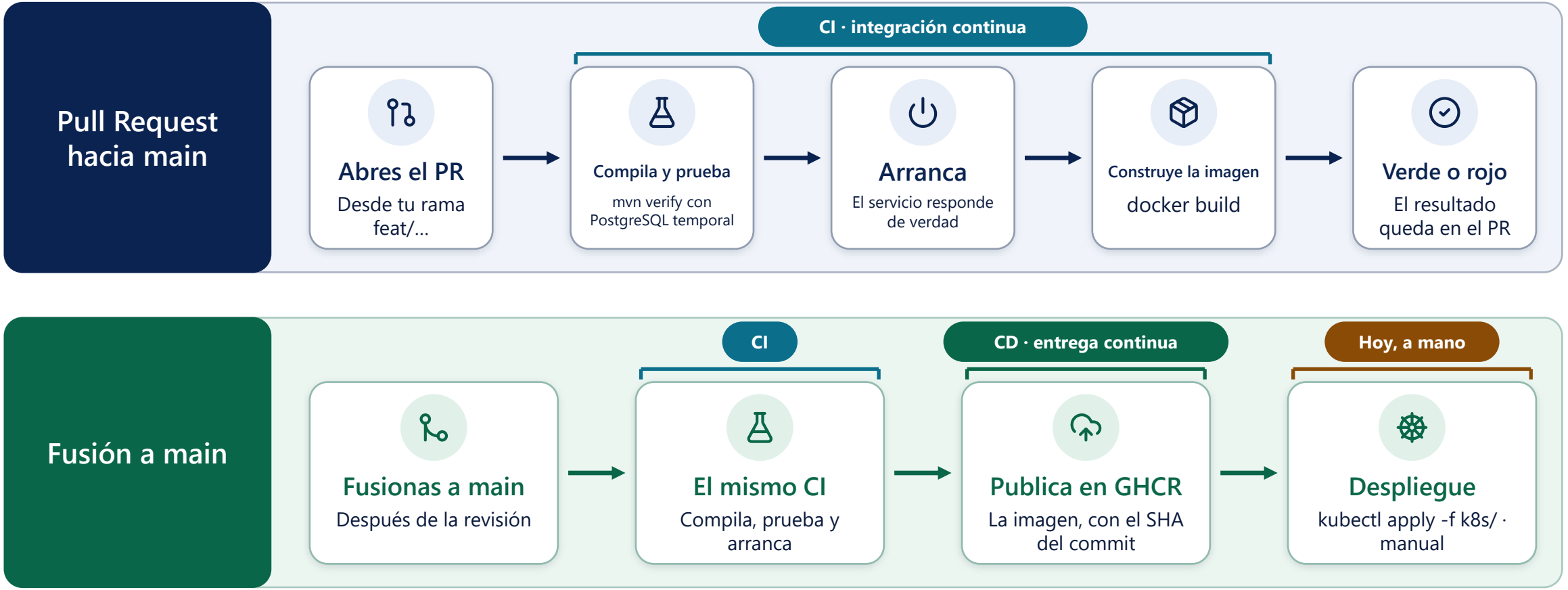
Demo

Desde el navegador.

¿Por qué importa? Probar o mostrar el sistema no depende de la computadora de nadie ni de la red de la oficina.

CI/CD: del push a la imagen publicada

CI comprueba cada cambio; CD deja la imagen lista para desplegar.



¿Por qué importa? Un cambio roto se detecta en el Pull Request, en minutos, antes de llegar a main.

CI/CD: del push a la imagen publicada · el código

.github/workflows/ci.yml

```yaml
on:
  push:
    branches: [ main ]   # fusión a main
  pull_request:          # cada PR
jobs:
  construir-probar-publicar:
    services:
      postgres: { image: postgres:17 }
    steps:
      - run: mvn -B verify
      - run: java -jar … # espera ver CAM-002
      - run: docker build … && docker push ghcr.io/…:${{ github.sha }}
```

Cómo trabaja Kubernetes: del archivo al Pod

Describimos en un archivo cómo queremos el sistema; Kubernetes hace lo necesario para que así sea, todo el tiempo.



Manifiesto YAML

Qué imagen, cuántas copias y qué variables.

kubectl apply

Se lo entrega al clúster.

Deployment

Crea los Pods con la imagen.

Service

Un nombre fijo para encontrarlos.

Ciclo de control

Compara lo que hay con lo pedido y corrige.

Piénsenlo así: Como un termostato: no le decimos cuándo prender el aire, le decimos la temperatura que queremos y él se encarga.

Los recursos de Kubernetes

Describimos el estado que queremos; Kubernetes lo mantiene.

ACCESO

- **Service** — Un nombre fijo por componente
- **port-forward** — La UI en localhost:8088

CÓMPUTO

- **5 Deployments** — UI, 2 servicios y 2 bases
- **initContainers** — Esperan a la base y al otro servicio
- **readinessProbe** — Sin tráfico hasta estar listo

CONFIGURACIÓN

- **Secret arka-postgres** — La clave de las bases
- **Variables de entorno** — DB_URL, NOTIFICACIONES_URL

DATOS

- **PersistentVolumeClaim** — Un disco por base: sobrevive al Pod

Los recursos de Kubernetes · el código

despliegue local

```
bash construir.sh            # imágenes de los 3 componentes
kubectl apply -f k8s/
kubectl get pods,svc,pvc
kubectl port-forward svc/arka-lite-ui 8088:8080
```

Cuando un Pod se cae

Kubernetes vuelve solo al estado que declaramos.

QUÉ FALLA

Se cae el Pod de solicitudes Un error, un nodo reiniciado o un kubectl delete.



Detecta

Falta 1 de 1 copias.

Recrea

Un Pod nuevo con la misma imagen.

Espera

initContainers: base y notificaciones.

Listo

La readinessProbe responde.

RESULTADO

Sin intervención manual Los eventos pendientes siguen en la base y se entregan.

DevOps: lo que aprendimos

LO QUE APRENDIMOS

1

comando para levantar todo

1

imagen por commit, rastreable

Docker y Kubernetes se complementan

Uno empaqueta; el otro ejecuta y cuida.

VALOR

Un mismo paquete, de la laptop al clúster.

La imagen es el entregable

Lo que prueba el pipeline es lo que se despliega.

VALOR

Sin sorpresas entre ambientes.

Configurar por variables

La cadena de conexión y las direcciones vienen del ambiente.

VALOR

Un solo artefacto para todos los ambientes.

Declarar el estado deseado

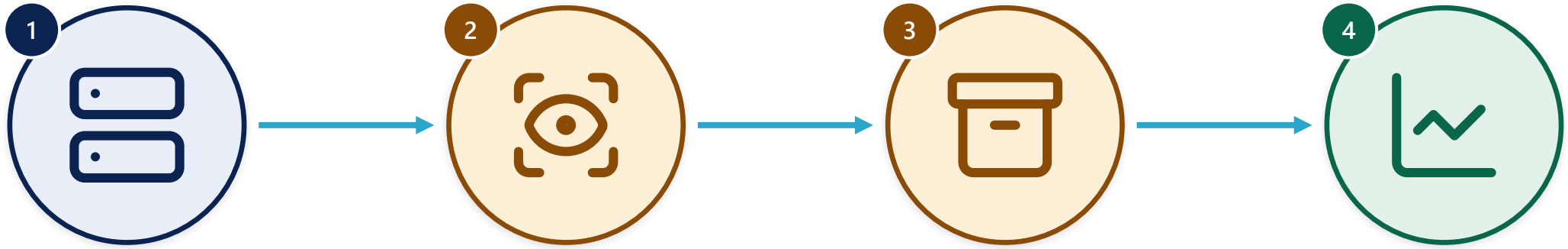
Deployments, pruebas de salud y un pipeline que arranca el servicio.

VALOR

El clúster se repara solo y los errores aparecen antes.

Todos los logs, en un solo lugar

Seguir un evento de un servicio a otro sin entrar a ningún contenedor.



Servicios

Logs JSON con traceId y eventId.

Promtail

Los recoge de cada contenedor.

Loki

Los guarda, etiquetados por servicio.

Grafana

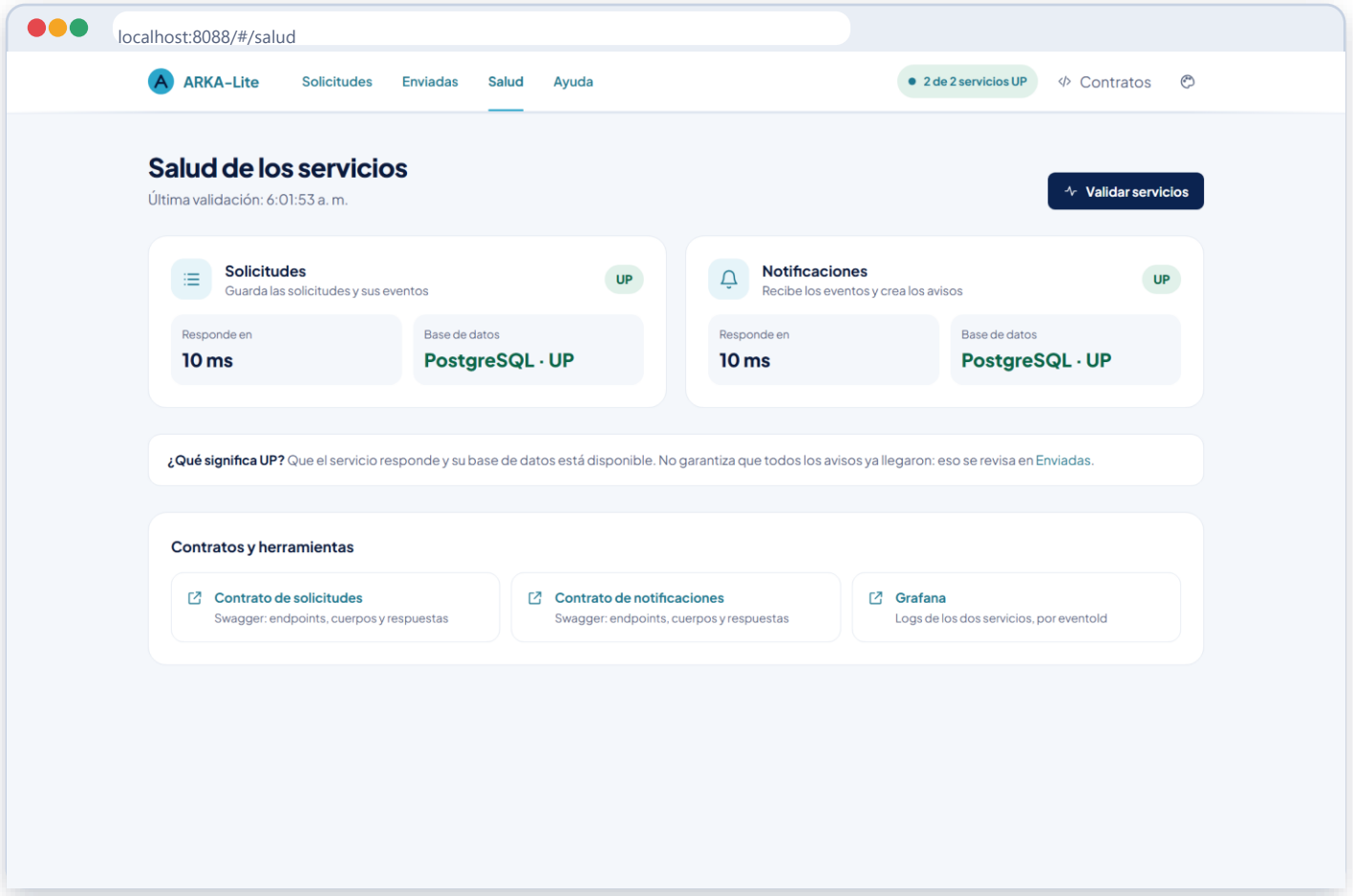
Una búsqueda para todo el sistema.

Piénsenlo así: Como la caja negra de un avión: registra lo que pasó, para revisarlo después.

Salud de cada servicio

Cada servicio responde: ¿estás bien?

- Actuator expone health, info y métricas
- La salud revisa la conexión con la base
- Kubernetes y la UI la consultan
- UP no garantiza que todos los avisos llegaron



Observabilidad: lo que aprendimos

LO QUE APRENDIMOS

1

lugar para buscar todos los logs

8

contenedores observados

Logs estructurados desde el día uno

JSON con formato estándar, sin cambiar el código de negocio.

VALOR

Buscar por campo en lugar de leer texto.

Un identificador que cruce servicios

El traceId viaja en cada llamada y el eventId en cada log.

VALOR

Diagnosticar un fallo entre servicios en minutos.

Salud no es lo mismo que correcto

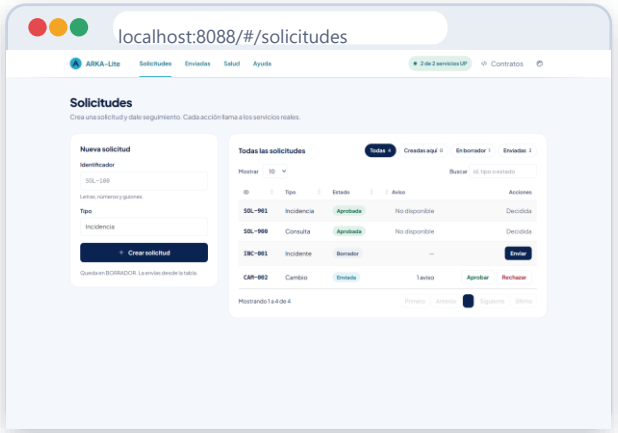
UP dice que responde; los pendientes dicen si entrega.

VALOR

Medir el resultado del negocio, no solo el proceso.

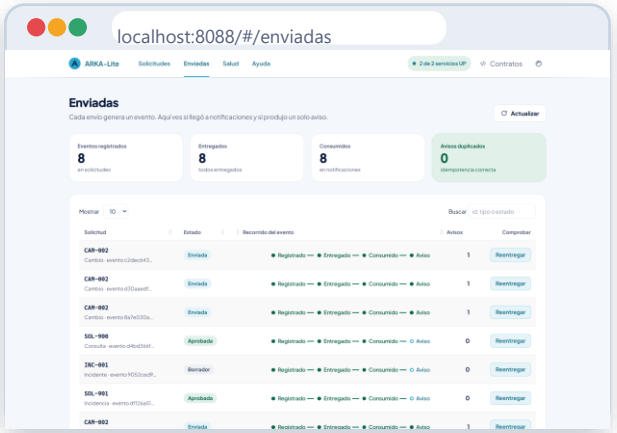
La UI: lo que contamos, en vivo

La forma de ver funcionando cada parte de la presentación.



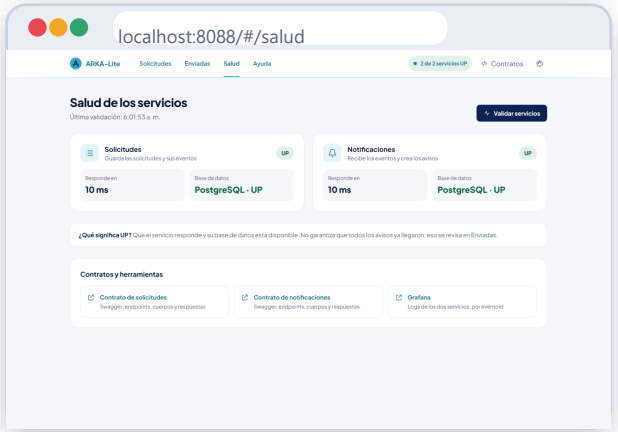
1 Solicitudes

El negocio: estados y errores 400, 404 y 409.



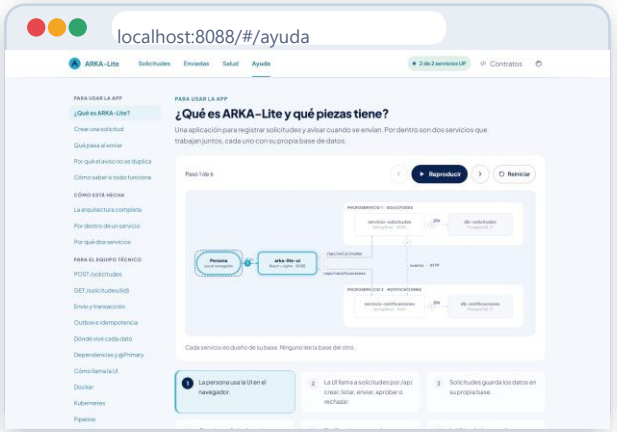
2 Enviadas

La interconexión: pendientes, entregados y Reentregar.



3 Salud

La observabilidad: UP o DOWN, Swagger y Grafana.



4 Ayuda

Diagramas paso a paso.

¿Por qué importa? Hace visible lo que normalmente no se ve: la bandeja de salida, los reintentos y la idempotencia.

En vivo: si notificaciones se cae

La persona no se entera; el aviso llega cuando el servicio vuelve, una sola vez.

QUÉ FALLA

notificaciones está caído docker compose stop servicio-notificaciones



Responde bien: solicitud ENVIADA y evento guardado.

3 intentos por ciclo; el evento sigue pendiente.

“Notificaciones no responde” y 1 en camino.

Mismo eventId: se crea el aviso.

RESULTADO

1 aviso, 0 perdidos Sin intervención y, al reentregar, sin duplicados.

Cómo se comporta cuando algo falla

Cada falla tiene una reacción prevista y algo visible en la UI.

QUÉ FALLA	CÓMO REACCIONA EL SISTEMA	QUÉ SE VE EN LA UI
notificaciones está caído	El evento queda pendiente y se reintenta cada 5 s	Aviso “Notificaciones no responde” · 1 en camino
El evento llega dos veces	ConsumoIdempotente lo reconoce y lo ignora	“El aviso no se duplicó” · 0 duplicados
Enviar dos veces o aprobar un borrador	El dominio rechaza la transición: 409	Diálogo con el motivo del 409
Faltan datos o el id ya existe	Se valida antes de guardar: 400 o 409	Diálogo “Faltan datos” o “Ese identificador ya existe”
Un POST tarda o se corta	La UI no lo repite sola: pudo haberse guardado	Error de tiempo agotado, sin reintento automático

Demostración: lo que aprendimos

LO QUE APRENDIMOS

5

fallas con reacción prevista

4

pantallas para ver el sistema por dentro

Diseñar para la falla

Cada falla probada tiene una reacción y un mensaje.

VALOR

Menos incidentes y menos llamadas a soporte.

Hacer visible lo asíncrono

Pendientes, entregados y consumidos en una pantalla.

VALOR

Explicar el sistema sin abrir el código.

Mostrar en vivo explica más

Botones para provocar fallas y ver la reacción.

VALOR

Material de capacitación para quien llegue al equipo.

El valor agregado

Prácticas que un equipo de tecnología puede reutilizar, en cualquier lenguaje.

PILAR	QUÉ CONSTRUIMOS	VALOR PARA EL EQUIPO
Datos consistentes	Transacción local, Outbox e idempotencia	0 avisos perdidos o duplicados ante caídas
Conexión segura	Cadena de conexión y claves fuera del código	Cambiar de servidor o de clave sin recompilar
Reproducibilidad	Docker, Compose y Kubernetes	1 comando y la misma imagen en todo ambiente
Calidad	Pruebas automáticas y pipeline en cada cambio	Errores detectados en minutos, antes de integrar
Trazabilidad	Logs centralizados, traceId e imagen por commit	Diagnóstico en minutos y versiones rastreables

Lo que pueden aplicar desde mañana

No hace falta reescribir nada: estas prácticas funcionan también con procedimientos almacenados.

PRÁCTICA	CÓMO SE VE EN SUS SISTEMAS DE HOY	QUÉ EVITA
Cadena de conexión fuera del código	Variables de entorno o un gestor de secretos en el servidor	Claves en el repositorio y recompilar por cambiar de servidor
Guardar y avisar en la misma transacción	Una tabla de pendientes que llena el procedimiento y un job que la entrega	Avisos perdidos cuando el otro sistema está caído
Idempotencia en lo que se reintenta	Una tabla de procesados con llave única por número de operación	Pagos, correos o registros duplicados
Un identificador que cruce sistemas	El mismo número en el log de la aplicación, del procedimiento y del otro sistema	Horas buscando un error entre sistemas
El mismo paquete en todos los ambientes	Contenedores para aplicaciones y bases de prueba	El clásico “en mi máquina funciona”

Lo que nos llevamos

LO QUE APRENDIMOS

0

avisos perdidos o duplicados

1

comando para levantar todo

Diseñar para la falla

Guardar primero, reintentar e ignorar duplicados.

VALOR

Sistemas que se recuperan solos.

Automatizar la confianza

Pruebas, pipeline, imágenes y estado declarado.

VALOR

Cambiar rápido sin miedo a romper.

Hacer visible el sistema

Logs, trazas, salud, Swagger y la UI.

VALOR

Diagnosticar y explicar en minutos.

A ARKA-Lite

Gracias

Equipo arka-litos

Proyecto final · Upskilling Backend Java

